

SLM autónomo

**Un Llama de 4B que aprende solo,
con la API de OpenAI como profesor y juez**

Código Python en un entorno virtual local • checkpoints • registros de uso y de
entrenamiento • dashboard de monitoreo — explicado paso a paso



Ahmad Armoush • Octubre 2026

Ruta: de la idea al modelo entrenado y monitoreado

- | | | |
|----|-------------------------|---|
| 01 | La idea y las piezas | por qué un SLM, destilación, el modelo de 4B, LoRA, hardware |
| 02 | Instalación paso a paso | venv, .env con tu clave, prueba gratuita y lanzamiento |
| 03 | El ciclo autónomo | currículo adaptativo, profesor, entrenamiento, juez, decisión, parada |
| 04 | Registros y monitoreo | qué se registra, dashboard, corrida de demostración, costos |
| 05 | Operación y riesgos | usar y exportar el modelo, licencias, checklist |

Un modelo grande cobra por cada token; uno pequeño, entrenado por él, corre en tu equipo

El objetivo no es reemplazar al LLM grande en todo, sino especializar un SLM en un dominio acotado.

LLM por API (el profesor)

- Máxima calidad, sin infraestructura
- Costo variable: se paga cada token, siempre
- Los datos salen de tu red
- Latencia de red y dependencia del proveedor

SLM local (el estudiante)

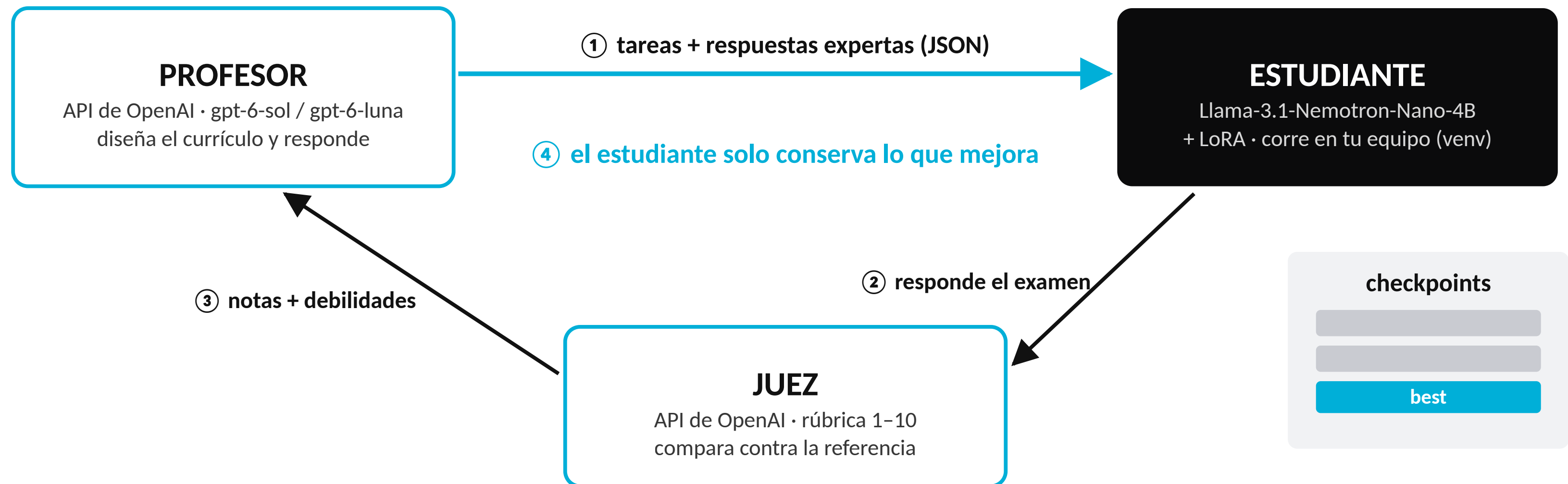
- Corre en tu GPU o Mac: costo marginal $\approx$ energía
- Privacidad: la inferencia no sale del equipo
- Control total de versión, latencia y disponibilidad
- Pero: sabe menos de tu dominio... hasta que lo entrenas

La propuesta

- Usar el LLM grande **solo para enseñar** durante unas horas
- Dejar el conocimiento en un modelo pequeño que es tuyo
- Automatizar todo el ciclo: datos, entrenamiento, evaluación y decisión

■ Pagas la enseñanza una vez ($\approx$ US\$ 13 en la configuración por defecto) y luego operas el modelo localmente.

El profesor enseña, el juez califica y el estudiante solo conserva lo que mejora



- Es destilación de conocimiento en bucle cerrado: sin intervención humana entre rondas.

01

La idea y las piezas

Destilación de caja negra, el modelo estudiante, LoRA y el hardware necesario.

Destilación de caja negra: el estudiante imita respuestas, no probabilidades

La API solo entrega texto; por eso el aprendizaje se hace por SFT sobre las respuestas del profesor.

Destilación clásica

Hinton, Vinyals y Dean (2015): el estudiante imita las **probabilidades** del profesor (soft targets, temperatura).
Requiere acceso a los logits del profesor.

Imposible con una API cerrada: solo devuelve texto.

Destilación de caja negra

El estudiante imita las **respuestas** (secuencias) del profesor: ajuste fino supervisado sobre pares instrucción → respuesta generados por el LLM grande.

Kim y Rush (2016): sequence-level KD ·
Self-Instruct (2022) · Alpaca (2023) ·
Orca (2023).

Lo que agrega este proyecto

- Currículo que se adapta a las notas
- Juez LLM con examen fijo
- Promoción o reversión de checkpoints
- Presupuesto y parada automáticos

■ La calidad del estudiante queda acotada por la calidad y diversidad de los datos que genera el profesor.

El “Llama de 4B” es Llama-3.1-Nemotron-Nano-4B: arquitectura Llama 3.1, 4,5 B parámetros

Ficha	nvidia/Llama-3.1-Nemotron-Nano-4B-v1.1
Parámetros	4,51 mil millones (4.512.746.496)
Arquitectura	LlamaForCausalLM · Llama 3.1 · decoder denso
Capas · ancho	32 capas · hidden 3.072 · MLP 9.216
Atención	32 cabezas de consulta · 8 de clave/valor (GQA) · head_dim 128
Vocabulario · contexto	128.256 tokens · hasta 128K tokens
Origen	Llama 3.1 8B → poda + destilación (Minitron) → post-entrenamiento NVIDIA
Razonamiento	on/off por prompt de sistema: usamos “detailed thinking off”
Licencia	NVIDIA Open Model License + Llama 3.1 Community License · descarga libre

Alternativas (config.yaml)

meta-llama/Llama-3.2-3B-Instruct
3,2 B · requiere aceptar licencia y HF_TOKEN

meta-llama/Llama-3.2-1B-Instruct
para notebooks modestos

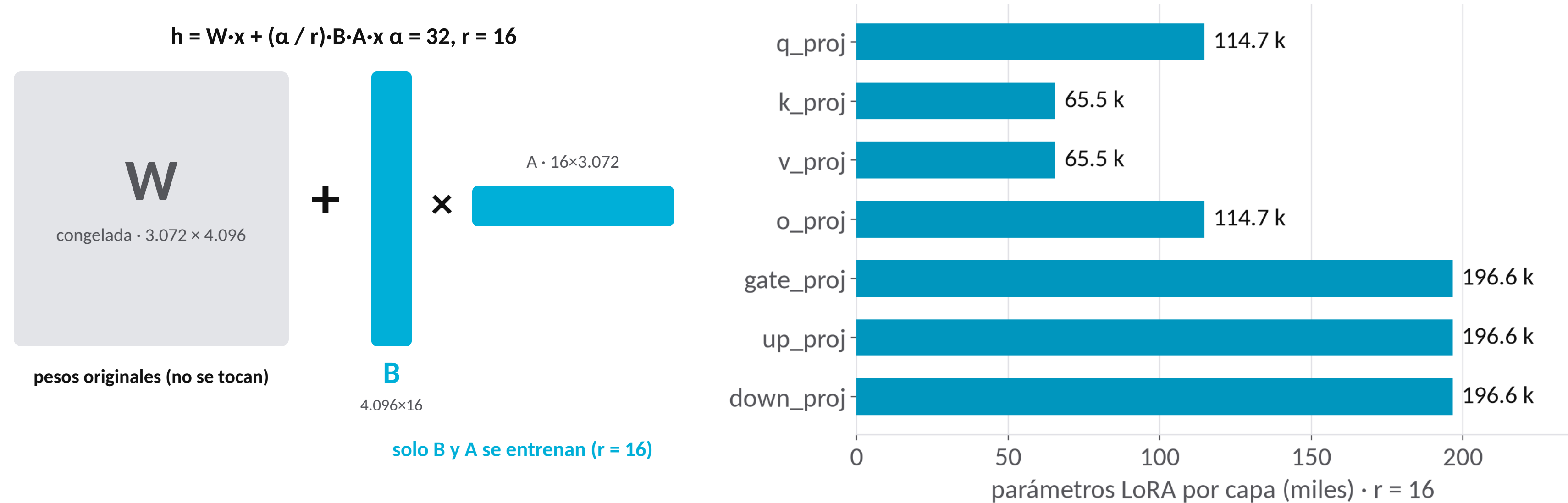
tiny-random-llama
3,7 M · se crea localmente para --simulate

Meta no publica un Llama oficial de 4B; el 4B de arquitectura Llama es el de NVIDIA.

- Es en sí un producto de destilación: un 8B comprimido a 4B. Nosotros lo especializamos un paso más.

LoRA entrena 30,4 M de parámetros: el 0,67 % del modelo

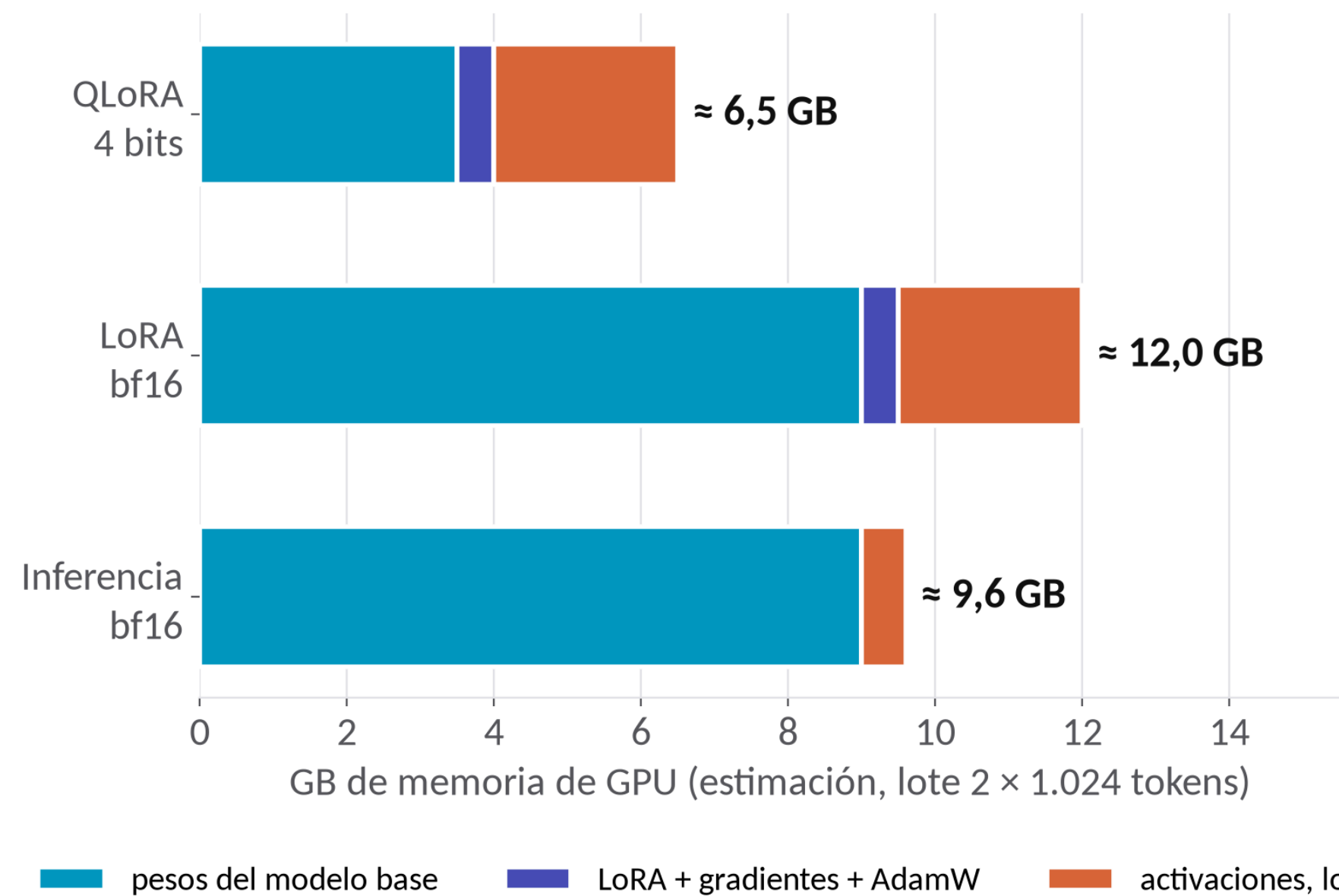
Se agregan matrices de bajo rango a las 7 proyecciones lineales de cada una de las 32 capas; el modelo base queda congelado.



- Un checkpoint pesa ≈ 120 MB en vez de 9 GB: guardar uno por ronda y revertir es barato.

Con QLoRA el modelo de 4B se entrena en una GPU de 12 GB

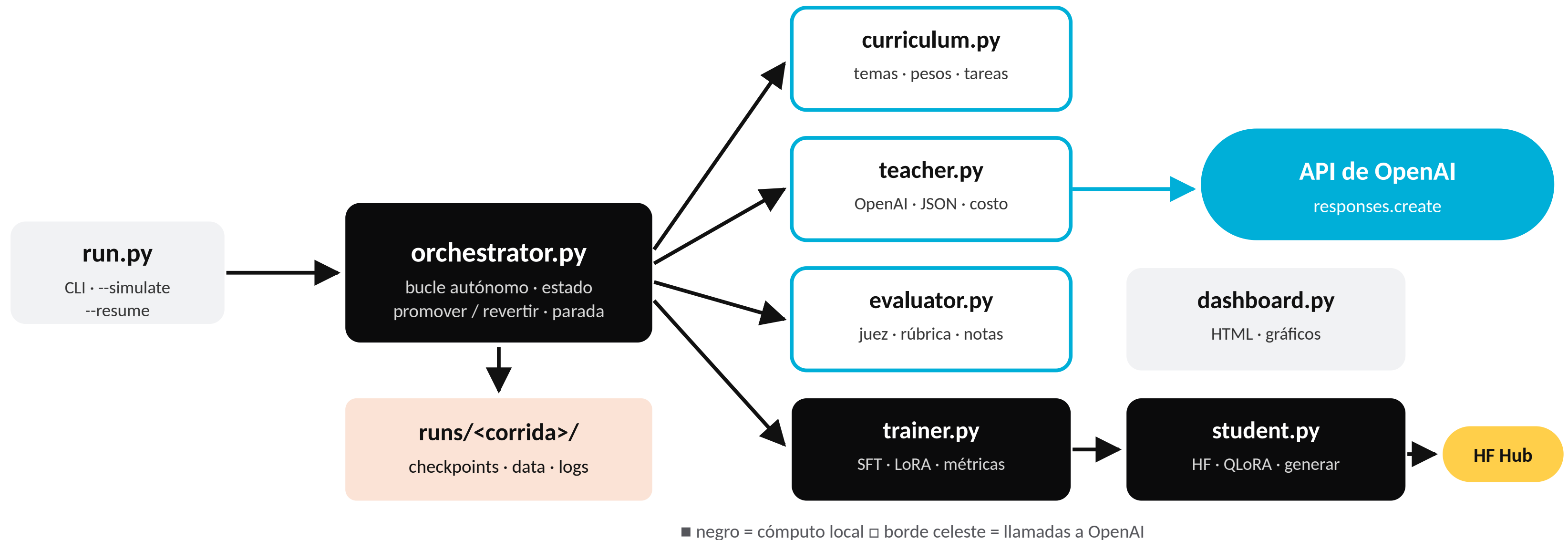
Estimación de memoria con gradient checkpointing, lote 2×1.024 tokens. Suma $\sim 0,5\text{--}1$ GB del contexto CUDA.



Equipo	Modo que elige el código
GPU NVIDIA 12–16 GB RTX 3060/4070, T4	QLoRA 4 bits (automático < 20 GB)
GPU NVIDIA ≥ 24 GB RTX 3090/4090, A10, L4	LoRA bf16
Mac Apple Silicon ≥ 24 GB	LoRA bf16 sobre MPS
CPU	solo --simulate

■ El código detecta CUDA, MPS o CPU y elige precisión y cuantización por sí solo (student.load_in_4bit: auto).

Arquitectura: ocho módulos, una responsabilidad cada uno



- El orquestador es el único que decide; los demás módulos son reemplazables (otro profesor, otro estudiante, otro juez).

02

Instalación paso a paso

Un entorno virtual, una clave en .env, una prueba gratuita y el lanzamiento.

Paso 1 • Crear el entorno virtual con un solo comando

```
# macOS / Linux
cd slm-autonomo
./setup_venv.sh
source .venv/bin/activate

# Windows (PowerShell)
.\setup_venv.ps1
.\.venv\Scripts\Activate.ps1
```

- 1 Verifica Python ≥ 3.10 y crea `.venv`
- 2 Instala PyTorch según el hardware: CUDA (NVIDIA), MPS (Mac) o CPU
- 3 Instala `requirements.txt`: transformers, peft, openai, bitsandbytes (solo NVIDIA), matplotlib, tensorboard...
- 4 Crea `.env` si no existe e imprime el dispositivo detectado

El modelo de 4B (≈ 9 GB) se descarga desde Hugging Face en la primera corrida real y queda en caché.

■ Todo vive dentro de `.venv`: nada se instala en el Python del sistema.

Paso 2 • La única edición obligatoria: tu clave en .env

```
# =====
# ÚNICO PASO OBLIGATORIO: pega tu clave aquí
# =====
OPENAI_API_KEY=sk-...

# ---- Opcional (ya vienen por defecto) ----
# OPENAI_MODEL_TEACHER=gpt-6-sol
# OPENAI_MODEL_JUDGE=gpt-6-sol
# OPENAI_MODEL_CURRICULUM=gpt-6-luna
# OPENAI_REASONING_EFFORT=low
# BUDGET_USD=20
# HF_TOKEN= (solo modelos meta-llama/*)
```

Seguridad de la clave

- .env está en .gitignore
- La clave nunca se escribe en runs/: el snapshot de configuración la excluye
- Sin clave, el programa se detiene con instrucciones claras

¿Otro proveedor compatible?

OPENAI_BASE_URL permite apuntar a Azure OpenAI u otro endpoint compatible con la API.

- Modelos, presupuesto y estilo de API se cambian en .env sin tocar código.

Paso 3 • Probar gratis con --simulate antes de gastar un dólar

En una máquina de 2 núcleos sin GPU, la corrida completa de 8 rondas tomó menos de un minuto.

```
$ python run.py --simulate
Corrida sim-... • dispositivo cpu • SIMULACIÓN
Set de evaluación fijo creado: 24 preguntas
Nota base del estudiante (sin entrenar): 1.00/10
Ronda 1: currículum → Fundamentos 15, RAG 15, ...
Ronda 1: 89 ejemplos nuevos del profesor
Ronda 1: entrenada • pérdida 7.11 → 4.64
Ronda 1: nota 1.00 (Δ +0.00) → ACEPTADO
Ronda 2: nota 1.42 (Δ +0.42) → PROMOVIDO
...
Ronda 8: nota 8.58 (Δ +0.87) → PROMOVIDO
Dashboard: runs/sim-.../dashboard.html
```

Qué hace la simulación

- Profesor y juez **simulados** en local: sin red y sin costo
- Un Llama diminuto (3,7 M) creado en el momento
- Recorre **todo** el pipeline real: datos, LoRA, juez, checkpoints, logs y dashboard

Valida que tu instalación funciona. Sus notas miden solapamiento con la referencia, no calidad real.

- Si la simulación termina con “ESTADO: finished”, la instalación está lista para la corrida real.

Paso 4 • Lanzar el entrenamiento real y dejarlo correr

```
# terminal 1: entrenamiento autónomo
python run.py

# terminal 2: monitoreo en vivo
python monitor.py --watch 30 --open
tensorboard --logdir runs    # opcional

# detener ordenadamente
touch runs/<corrida>/STOP

# reanudar tras un corte o reinicio
python run.py --resume-last

# límites rápidos
python run.py --rounds 4 --budget 8
```

- 1 **Ronda 0:** taxonomía, examen fijo y nota base
- 2 **Rondas 1...N:** currículum → profesor → LoRA → juez → decisión
- 3 **Parada** automática por nota, paciencia, presupuesto, rondas, horas o STOP
- 4 **Resultado:** checkpoints/best, logs, usage_summary.json y dashboard.html

Cada etapa es idempotente: al reanudar se salta lo que ya está en disco (datos, checkpoints, evaluaciones).

■ Ctrl+C también es seguro: el estado se guarda y la corrida se retoma donde quedó.

config.yaml: diez perillas explican casi todo el comportamiento

Parámetro	Defecto	Efecto
curriculum.tasks_per_round	200	tareas nuevas que genera el profesor por ronda
evaluation.eval_size	40	preguntas del examen fijo (más = nota menos ruidosa)
training.epochs_per_round	2	pasadas sobre nuevos + repaso
training.learning_rate	2e-4	tasa de aprendizaje de LoRA (coseno con calentamiento)
training.replay_ratio	1.0	ejemplos antiguos por cada nuevo (anti-olvido)
lora.r / lora.alpha	16 / 32	rango y escala de los adaptadores
evaluation.min_delta	0,05	mejora mínima para promover un checkpoint
evaluation.rollback_tolerance	0,30	caída máxima tolerada antes de revertir
autonomy.target_score · patience	8,5 · 3	nota objetivo · rondas sin mejora para parar
autonomy.budget_usd · max_rounds	20 · 8	tope de gasto en la API · máximo de rondas

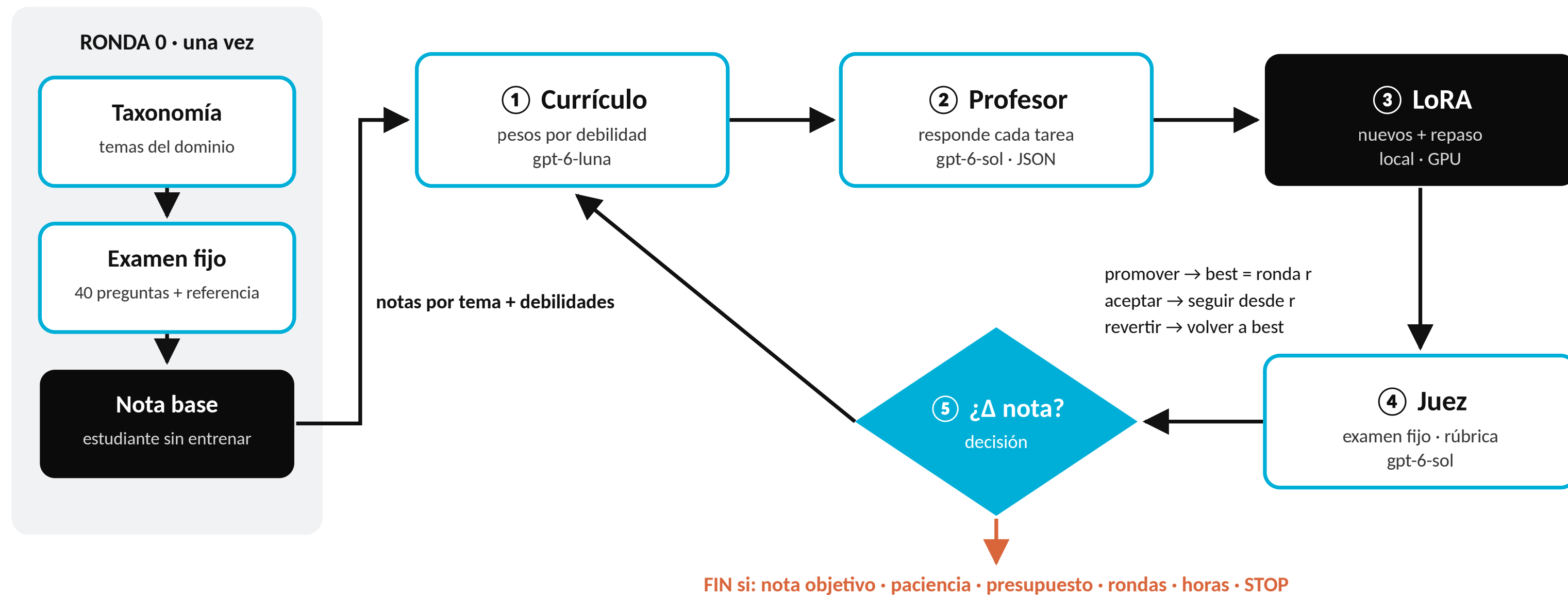
- El dominio también se define ahí: descripción, idioma, estilo y lista de temas (o vacía para que el profesor la proponga).

03

El ciclo autónomo

Cinco etapas por ronda y una regla de decisión que protege al mejor modelo.

Una ronda = currículum → profesor → LoRA → juez → decisión



Ronda 0: un examen fijo que nunca se usa para entrenar

Qué se construye

- Temas: los de config.yaml o propuestos por el profesor
- 40 preguntas estratificadas por tema
- Respuesta de referencia del profesor para cada una
- Se guarda en `data/eval_set.jsonl`

Por qué fijo y separado

- Las notas de todas las rondas son comparables
- Las preguntas se registran en el deduplicador: ninguna tarea de entrenamiento puede parecerse $\geq 85\%$
- Sin fuga de examen $\rightarrow$ la mejora es generalización

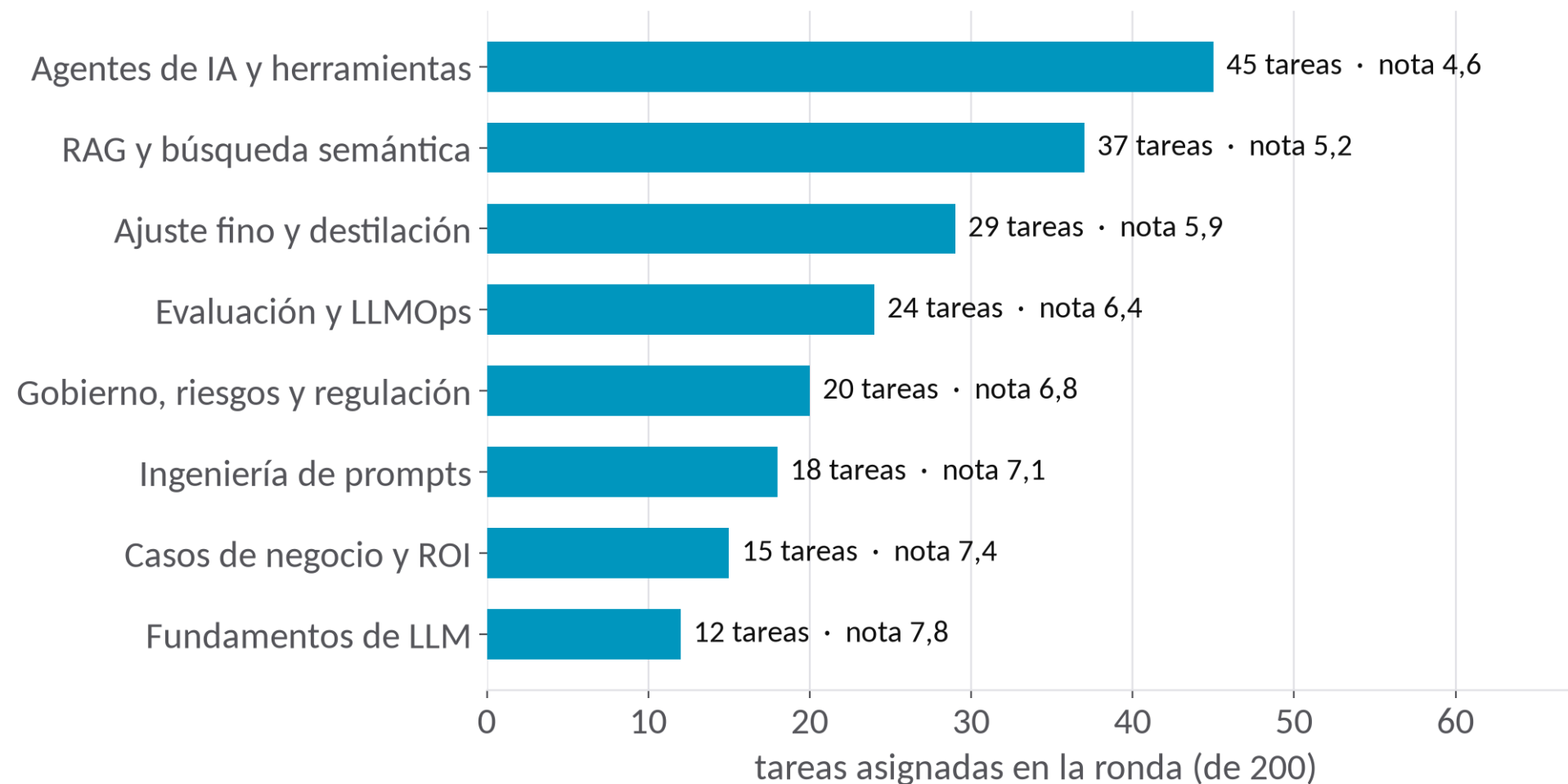
Nota base

El estudiante responde el examen **antes** de entrenar (LoRA con $B = 0 \equiv$ modelo original). Ese checkpoint `round_000` es el primer “mejor”.

■ Sin línea base no hay forma de saber si el entrenamiento ayudó o dañó al modelo.

Currículo adaptativo: más práctica donde el estudiante falla

Ejemplo con 8 temas y 200 tareas: Agentes (nota 4,6) recibe 45 tareas; Fundamentos (7,8) recibe 12.



Regla

$$\text{peso}_t \propto \max(0,05 ; ((10 - \text{nota}_t) / 10)^{1,5})$$

$\gamma = 1,5$ concentra la práctica en lo débil; el piso de 5 % evita abandonar temas.

Además, la retroalimentación del juez sobre las peores respuestas entra al prompt del profesor: “incluye tareas que ejerciten estas debilidades”.

- El reparto se hace por restos mayores: suma exacta, sin temas olvidados.

El profesor responde con salidas estructuradas y se descarta lo dudoso

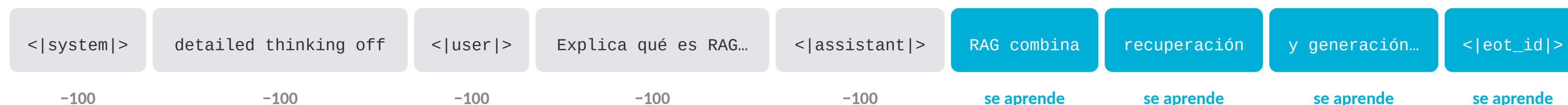
```
# teacher.py · Responses API + JSON Schema estricto
client.responses.create(
    model="gpt-6-sol",
    instructions=profesor_sistema,
    input=tarea["instruction"],
    max_output_tokens=2000,
    reasoning={"effort": "low"},
    text={"format": {"type": "json_schema",
        "name": "respuesta", "strict": True,
        "schema": {... "answer": string ...}}})
```

- 1 **8 llamadas en paralelo** con reintentos y backoff ante 429/5xx
- 2 **Capacidades autodetectadas:** si un modelo rechaza reasoning o json_schema, se desactiva solo para él
- 3 **Deduplicación:** trigramas de caracteres, Jaccard $\geq 0,85 \rightarrow$ descarte
- 4 **Filtro de calidad:** largo de respuesta entre 80 y 2.600 caracteres
- 5 **Respuesta truncada:** se reintenta con el doble de tokens

■ Cada llamada, incluso las fallidas, queda en openai_usage.jsonl con tokens, costo y latencia.

Entrenamiento: el modelo aprende la respuesta, nunca la pregunta

Formato de chat nativo del modelo (plantilla de Nemotron) + fin de turno `<|eot_id|>`.



Máscara de etiquetas

Los tokens del prompt llevan etiqueta -100: la pérdida solo mide la respuesta. El modelo aprende a **responder**, no a repetir preguntas.

Mezcla anti-olvido

Por cada ejemplo nuevo, uno de rondas anteriores (replay 1:1). Se parte del checkpoint vigente, no desde cero.

Optimización

AdamW · lr 2e-4 coseno con 5 % de calentamiento · lote efectivo 16 · recorte de gradiente 1,0 · gradient checkpointing.

- Por paso se registran pérdida, tasa de aprendizaje, norma del gradiente, tokens/s y memoria.

El juez califica con rúbrica y respuesta de referencia

Rúbrica (1–10)

- **Corrección:** exactitud; penaliza invenciones
- **Compleitud:** cubre lo esencial de la referencia
- **Claridad:** estructura y utilidad
- **Idioma y formato:** responde en el idioma pedido
- **Nota global** + una oración de retroalimentación

Sesgos y cómo se mitigan

- Largo: instrucción explícita de no premiarlo
- Variabilidad: examen fijo y mismo modelo juez en todas las rondas
- Juez sin ancla: siempre recibe la respuesta de referencia
- Lo que no se califica no entra al promedio

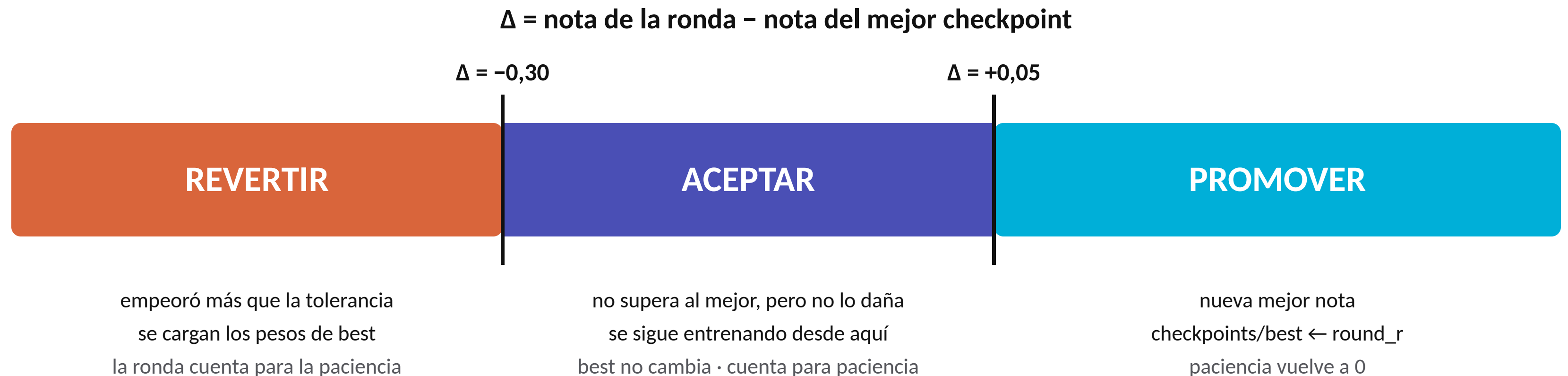
Salidas

- nota media, mediana y desviación
- tasa de aprobación (≥ 7)
- nota por tema, criterio y dificultad
- peores respuestas → próximo currículo

■ LLM-as-a-judge escala la evaluación, pero conviene auditar a mano una muestra de eval_details/ cada tanto.

Promover, aceptar o revertir: la regla que protege al mejor modelo

La zona “aceptar” evita estancarse: en nuestras pruebas simuladas, con solo promover o revertir el bucle se estancaba.



- Cualquiera sea la decisión, el checkpoint de la ronda queda guardado con su meta.json: nada se pierde.

Seis condiciones detienen el bucle sin intervención humana

Nota objetivo

best $\geq$ 8,5

Paciencia

3 rondas sin promover

Presupuesto

gasto $\geq$ US\$ 20

Máx. rondas

8

Máx. horas

24

Archivo STOP

en la carpeta de la corrida

Robustez: estado atómico (state.json) · reanudación idempotente · reintentos con backoff · detección de capacidades por modelo · Ctrl+C seguro · el monitoreo nunca detiene el entrenamiento.

- El presupuesto se verifica antes de cada llamada: el gasto solo puede superar el tope por las llamadas ya en curso (máx. 8 en paralelo).

04

Registros y monitoreo

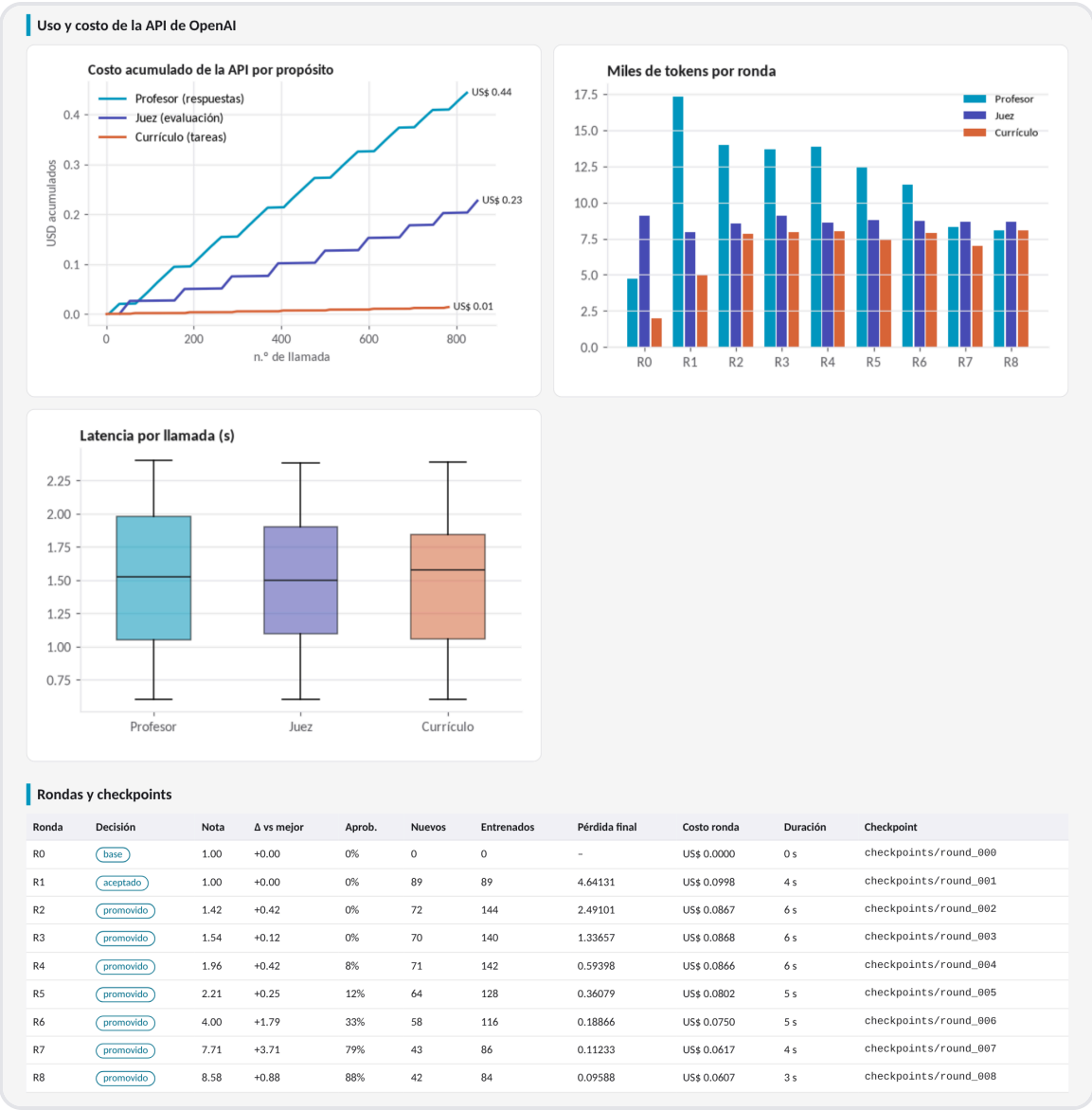
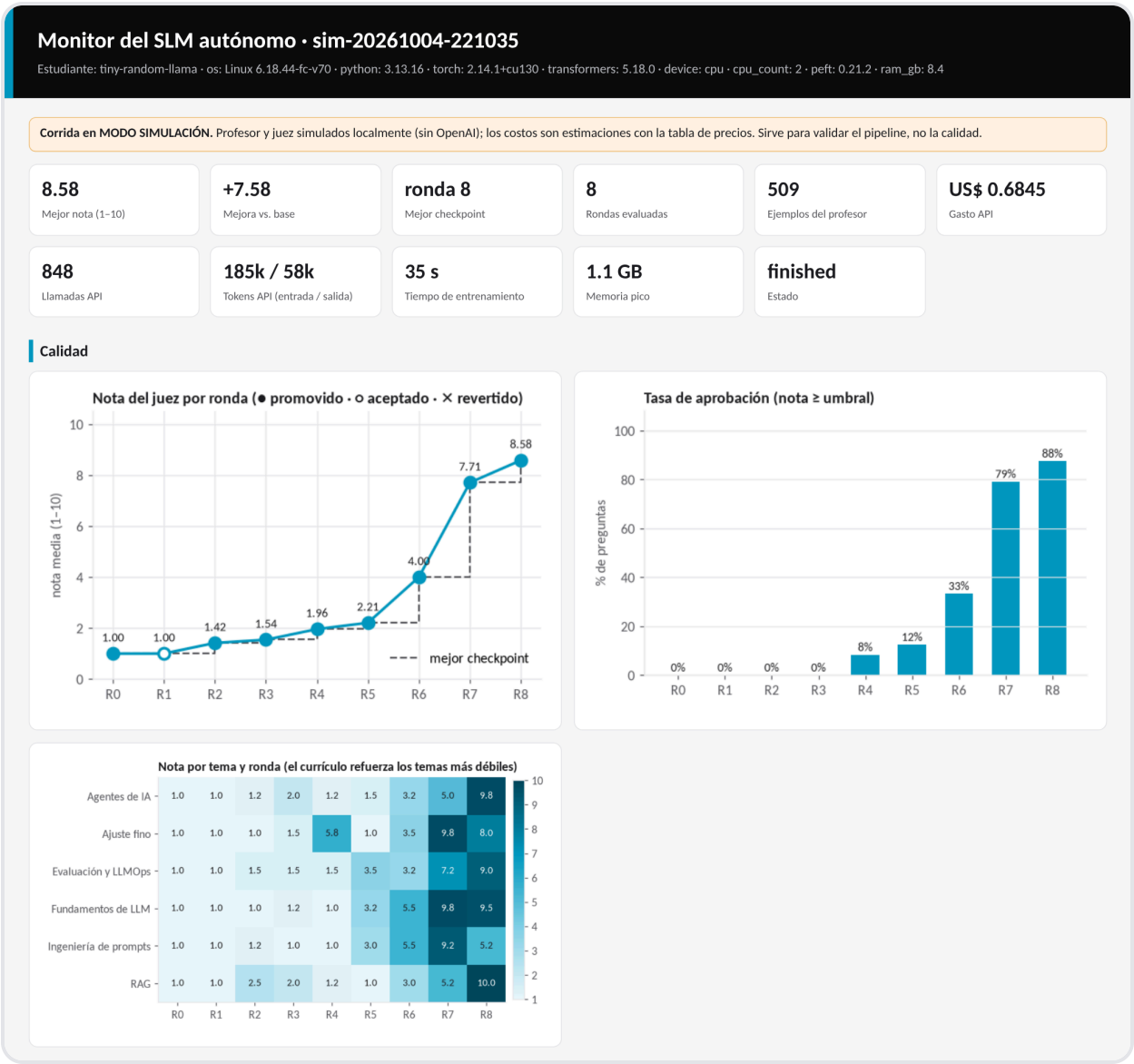
Qué queda en disco, cómo se ve y qué dice una corrida de demostración.

Cada llamada, cada paso y cada ronda quedan registrados

runs/<corrida>/	Contenido
checkpoints/round_XXX/	adaptador LoRA + meta.json (nota, decisión, métricas)
checkpoints/best/	copia del mejor checkpoint
data/round_XXX.jsonl · eval_set.jsonl	datos del profesor por ronda · examen fijo
logs/openai_usage.jsonl	cada llamada: propósito, modelo, tokens (entrada, caché, salida, razonamiento), USD, latencia, estado
logs/train_metrics.jsonl	cada paso: pérdida, lr, norma del gradiente, tokens/s, memoria
logs/eval_metrics.jsonl · eval_details/	notas por ronda, tema, criterio y dificultad · detalle por pregunta
logs/rounds.jsonl · events.jsonl · run.log	resumen por ronda · bitácora · consola
usage_summary.json · state.json · system.json	uso agregado · estado reanudable · hardware y versiones
dashboard.html · tensorboard/	monitoreo autocontenido · curvas en TensorBoard

- **Formato JSONL:** una línea por evento, legible en Excel, pandas o cualquier herramienta de BI.

El dashboard reúne calidad, entrenamiento y costo en una página

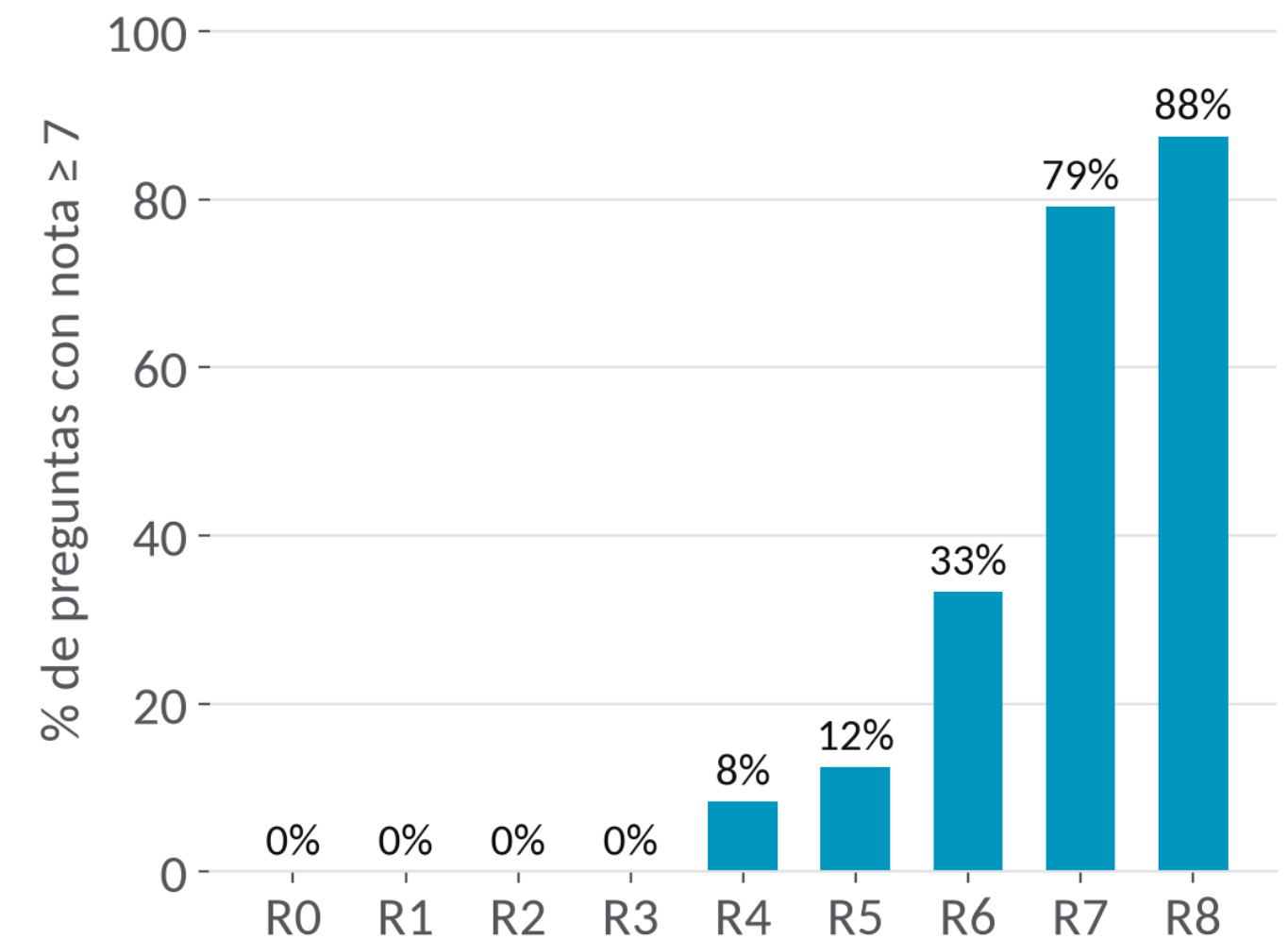
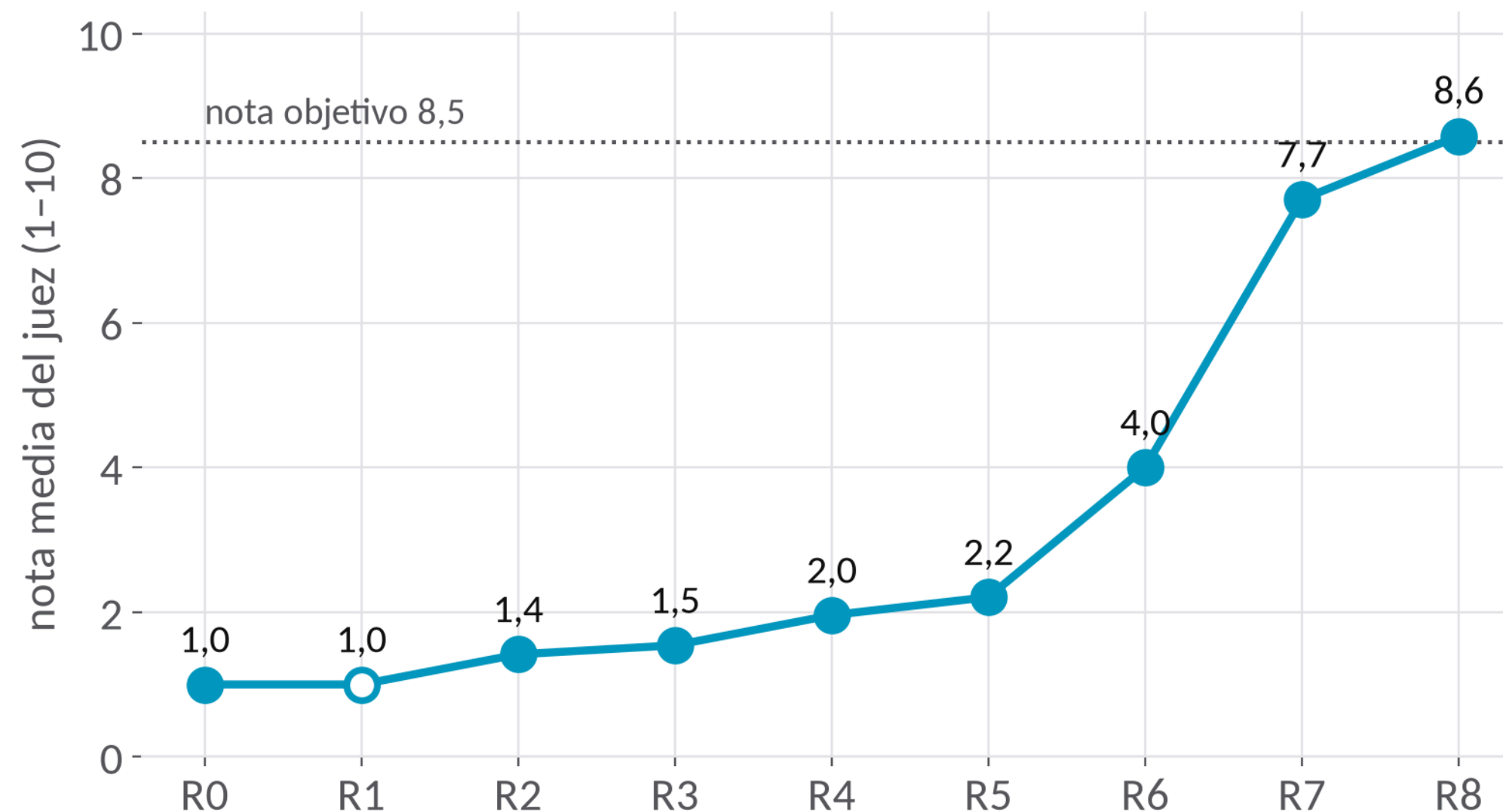


- 1 KPIs:** mejor nota, mejora vs. base, checkpoint, gasto, tokens, tiempo
- 2 Calidad:** nota por ronda, aprobación y mapa de calor por tema
- 3 Entrenamiento:** pérdida, tasa de aprendizaje, tokens/s
- 4 API:** costo acumulado, tokens por ronda, latencias
- 5 Tablas:** rondas y checkpoints, peores respuestas, bitácora

■ python monitor.py --watch 30 lo regenera mientras entrena; funciona sin internet.

Corrida de demostración: de 1,0 a 8,6 en ocho rondas

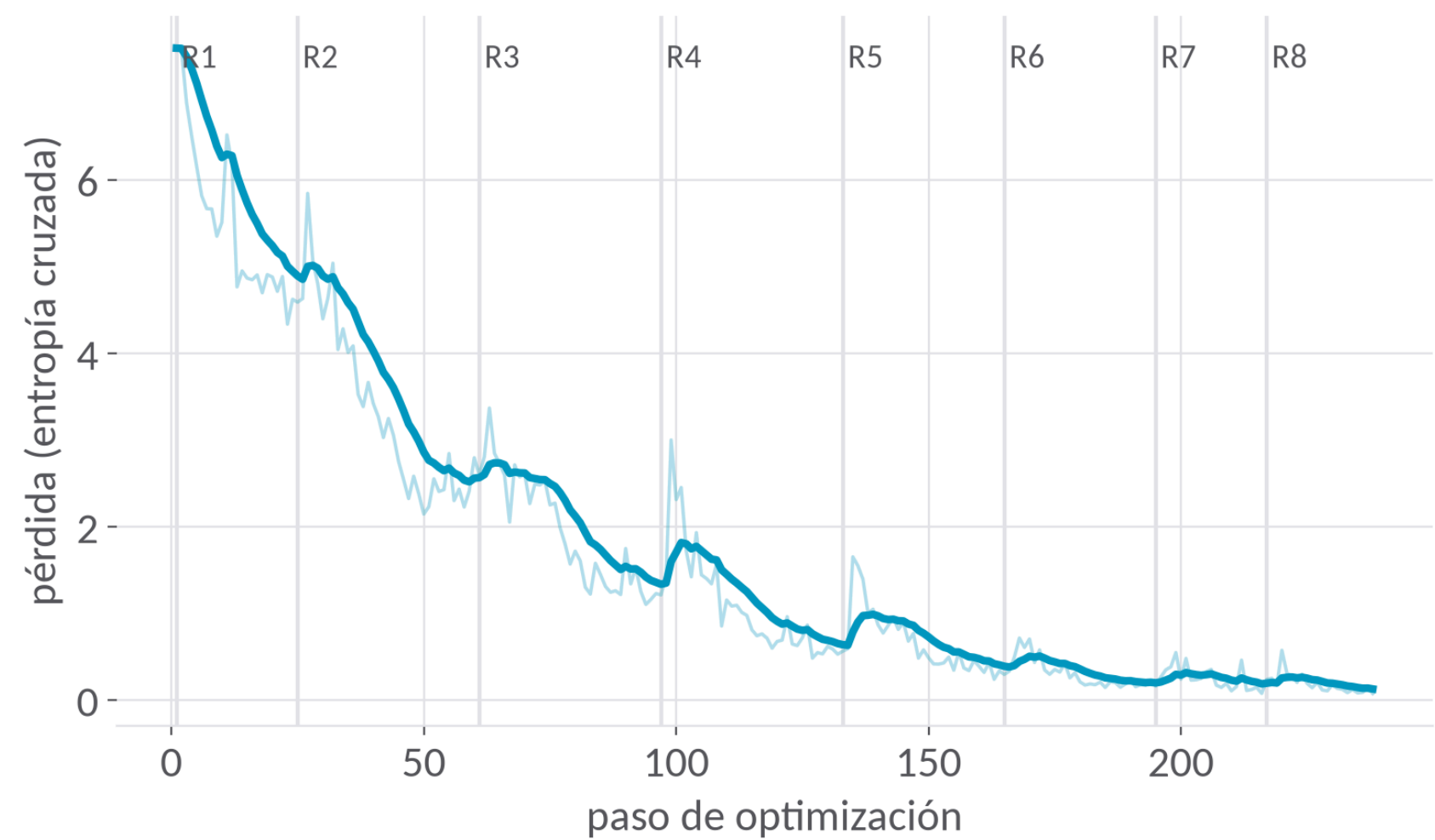
Llama diminuto (3,7 M parámetros) con profesor y juez simulados; ● promovido • ○ aceptado. Valida el pipeline, no la calidad de un modelo real.



- El bucle se detuvo solo al alcanzar la nota objetivo en la ronda 8, con 509 ejemplos del profesor.

La pérdida cae ronda a ronda y las notas suben tema por tema

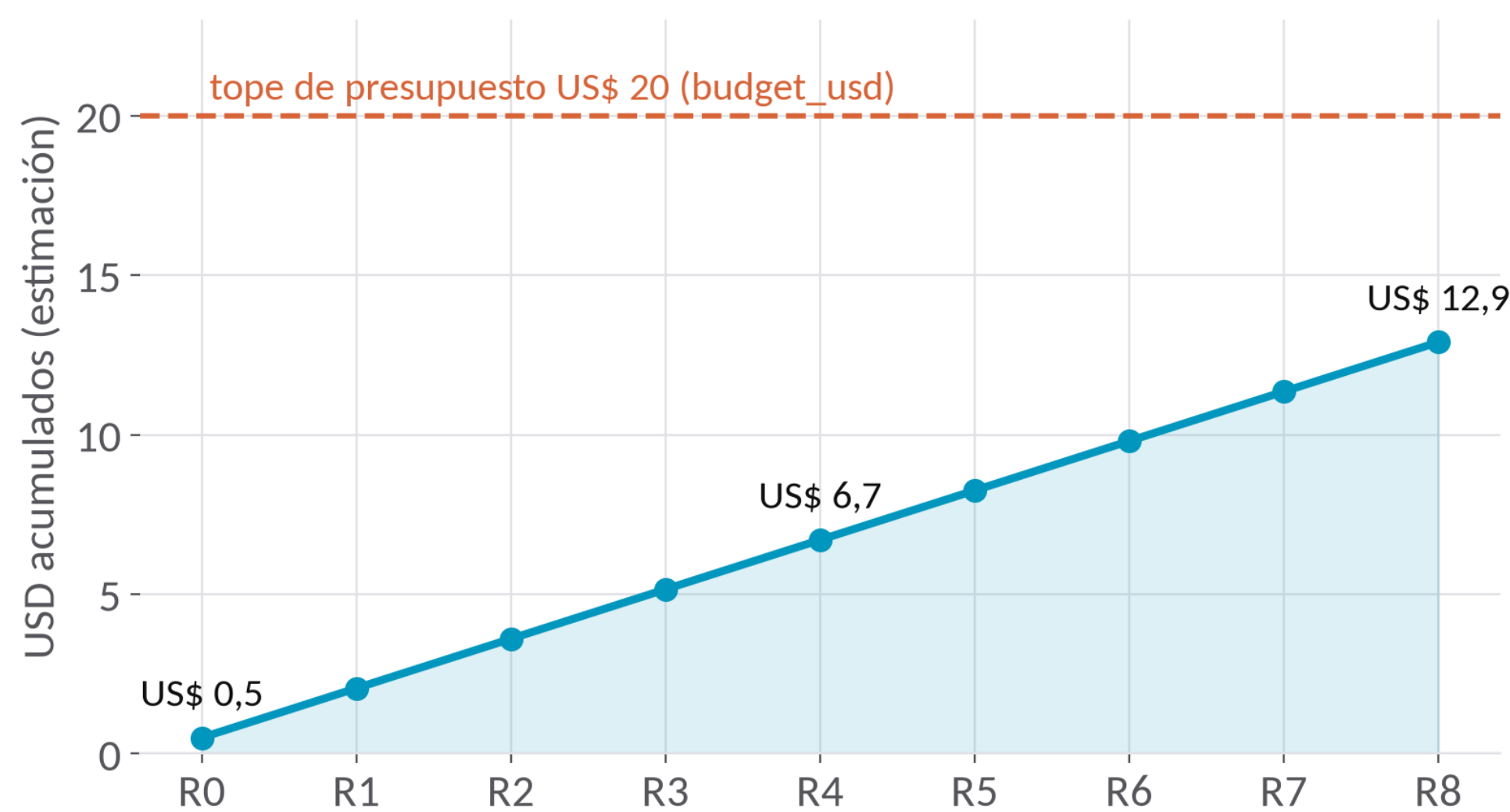
Izquierda: pérdida por paso (media móvil) con el inicio de cada ronda. Derecha: nota media por tema y ronda.



Agentes de IA	1,0	1,0	1,2	2,0	1,2	1,5	3,2	5,0	9,8
Ajuste fino	1,0	1,0	1,0	1,5	5,8	1,0	3,5	9,8	8,0
Evaluación y LLMOps	1,0	1,0	1,5	1,5	1,5	3,5	3,2	7,2	9,0
Fundamentos de LLM	1,0	1,0	1,0	1,2	1,0	3,2	5,5	9,8	9,5
Ingeniería de prompts	1,0	1,0	1,2	1,0	1,0	3,0	5,5	9,2	5,2
RAG	1,0	1,0	2,5	2,0	1,2	1,0	3,0	5,2	10,0
	R0	R1	R2	R3	R4	R5	R6	R7	R8

■ Una pérdida que baja sin que suba la nota del juez es la señal clásica de sobreajuste.

Una corrida real de 8 rondas cuesta del orden de US\$ 13



Por ronda	Llamadas	USD
Profesor (gpt-6-sol)	200	1,30
Juez (gpt-6-sol)	40	0,24
Currículo (gpt-6-luna)	≈ 24	0,01
Total ronda	≈ 264	1,55

Supuestos: ≈ 260 tokens de entrada y 600 de salida por respuesta; ≈ 1.050 / 380 por juicio. Precios de lista a oct-2026: Sol US\$ 2 / 10 y Luna US\$ 0,10 / 0,50 por millón de tokens (entrada / salida).

Todo con gpt-6-luna: ≈ US\$ 0,74 por 8 rondas, a cambio de un profesor menos capaz.

- El 84 % del gasto es el profesor escribiendo respuestas: ahí está la palanca para ahorrar.

Seis señales de alerta y qué hacer con cada una

Señal en el monitoreo	Causa probable	Acción
La pérdida baja, la nota no sube	sobreajuste o datos poco diversos	bajar épocas; subir tasks_per_round; revisar dedup
Muchas reversiones seguidas	lr alto u olvido; examen ruidoso	bajar learning_rate; subir eval_size o replay_ratio
Un tema no mejora	tareas mal formuladas para ese tema	revisar data/ y eval_details/ de ese tema
Costo por ronda sube	respuestas truncadas, reintentos, razonamiento alto	ver status en openai_usage; bajar reasoning_effort
tokens/s muy bajos	corre en CPU o sin bf16	revisar system.json y el dispositivo detectado
Memoria agotada (OOM)	lote o secuencia grandes	batch_size 1, max_seq_len 768, QLoRA

■ Todas las acciones son cambios en config.yaml y se aplican al reanudar la corrida.

05

Operación y riesgos

Usar el modelo entrenado, licencias y lista de verificación.

Usar el modelo: conversar, comparar, exportar y servir

```
# conversar con el mejor checkpoint
python chat.py

# comparar base vs. entrenado
python chat.py --compare

# fusionar base + LoRA → modelo HF completo
python export.py
#   exports/<corrida>-rX/

# servir (opcional)
vllm serve exports/<corrida>-rX
# o convertir a GGUF (llama.cpp) → Ollama / LM
Studio
```

Dos formas de distribuirlo

- Solo el adaptador (≈ 120 MB) + modelo base desde HF
- Modelo fusionado (≈ 9 GB bf16) listo para servir

Con `--compare` se ve lado a lado lo que el modelo aprendió: es la mejor demostración para un comité.

- El resultado es un modelo Hugging Face estándar: no queda atado a este repositorio.

Riesgos: licencias, datos y un juez que también se equivoca

Licencias

- Términos de OpenAI: restringen usar sus salidas para desarrollar modelos que **compitan** con OpenAI
- Estudiante: NVIDIA Open Model License + Llama 3.1 Community License (“Built with Llama”)
- Revisar con el equipo legal antes de uso comercial

Datos

- Los prompts del profesor salen a la API: evita datos personales o confidenciales
- Revisa una muestra de data/ antes de publicar el modelo
- El estudiante hereda sesgos y errores del profesor

Evaluación

- El juez es otro LLM: audita a mano 20–30 respuestas
- Agrega preguntas reales de usuarios al examen
- Una nota alta en el examen no garantiza seguridad: prueba casos adversos

■ La autonomía cubre el entrenamiento; la decisión de poner el modelo en producción sigue siendo humana.

Lista de verificación para la primera corrida real

- ✓ Python ≥ 3.10 y `./setup_venv.sh` sin errores
- ✓ Clave en `.env` (`OPENAI_API_KEY`)
- ✓ `python run.py --simulate` termina en “finished”
- ✓ Dominio, idioma y temas ajustados en `config.yaml`
- ✓ Precios verificados en openai.com/api/pricing y `budget_usd` definido
- ✓ Espacio en disco: ≈ 10 GB para el modelo + ≈ 120 MB por checkpoint
- ✓ `python run.py` y `monitor.py --watch` en paralelo
- ✓ Auditar `eval_details/` del mejor checkpoint
- ✓ `python chat.py --compare` para la demostración
- ✓ `python export.py` para distribuir

Cinco ideas para llevarse

Destilar

un LLM grande enseña y un SLM local aprende, por SFT sobre sus respuestas

Medir

examen fijo + juez con rúbrica antes y después de cada ronda

Adaptar

el currículo se mueve hacia los temas con peor nota

Proteger

promover, aceptar o revertir: el mejor checkpoint nunca se pierde

Registrar

cada token, paso y decisión queda en disco y en el dashboard

Referencias

- Hinton, Vinyals y Dean (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531
- Kim y Rush (2016). Sequence-Level Knowledge Distillation. EMNLP. arXiv:1606.07947
- Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685
- Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314
- Wang et al. (2022). Self-Instruct. arXiv:2212.10560 • Taori et al. (2023). Stanford Alpaca
- Mukherjee et al. (2023). Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707
- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685
- Sreenivas et al. (2024). LLM Pruning and Distillation in Practice: The Minitron Approach. arXiv:2408.11796
- NVIDIA (2025). Llama-3.1-Nemotron-Nano-4B-v1.1 • huggingface.co/nvidia/Llama-3.1-Nemotron-Nano-4B-v1.1
- OpenAI. API Pricing (openai.com/api/pricing) y Terms of Use • consultados en octubre de 2026

Gracias

`python run.py --simulate` → `python run.py`

Repositorio: `slm-autonomo` · README.md con todos los comandos